Ders 6 Genişletilmiş Notu: Command, Iterator, Memento, Observer, Strategy ve State Kalıpları

Ders 6 Genişletilmiş Notu: Command, Iterator, Memento, Observer, Strategy ve State Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Command Kalıbı: Amaç ve GoF Örneği
- 2. Command Kalıp Yapısı
- 3. Örnek Gerçekleme: Akıllı Ev Kumandası
- 4. Iterator Kalıbı: Amaç ve İki Çözüm
- 5. Iterator Kalıp Yapısı ve Gerçekleme Ayrıntıları
- 6. Örnek Gerçekleme: İki Restoranın Menüsü
- 7. Memento Kalıbı
- 8. Örnek Gerçekleme: İç Sınıf ile Memento
- 9. Observer Kalıbı
- 10. Observer Gerçekleme Ayrıntıları: Pull ve Push Modelleri
- 11. Örnek Gerçekleme: Müşteri Kaydında Gözleyiciler
- 12. Strategy Kalıbı
- 13. Strategy ile Builder Karşılaştırması
- 14. Örnek Gerçekleme: Uluslararası Vergi Hesabı
- 15. State Kalıbı
- 16. Örnek Gerçekleme: Kişisel Bilgi Yöneticisinde Temiz ve Kirli Durum
- Örnek Sorular
- Dersin Toparlaması

1. Command Kalıbı: Amaç ve GoF Örneği

Ders, önceki derste başlanan davranışsal (behavioral) kalıplardan Command ile sürer. Hoca, dersteki örneklerden yalnız ikisini daha iyi olsun istediğini, bunlardan birinin Command örneği olduğunu söyledi; bu dönem için daha iyi bir örnek hazırlayamadığından basit örnekle devam etti.

1.1 Amaç

Kalıbın amacı, nesnelere giden istekleri de nesne haline getirmektir. Hoca bunu “metotları nesne yapmak” gibi düşünmeyi önerdi. İstekler nesne olunca bunları kuyruklamak, bir günlükte (log) tutmak ya da undo/redo gibi işlemler yapmak mümkün olur.

Herhangi bir GUI kitaplığıyla kod yazan bu durumu bilir: bir pencere (frame) hazırlanır, üst menüde “Dosya Aç” tıklanınca ne olacağı tanımlanır. “Dosya aç tıklanınca git şu nesnenin şu metodunu çalıştır” türünden kodları yürütmek için bu kalıp kullanılabilir. Bazı çatılar (framework) “şöyle bir nesnenin şöyle bir metodunu çağır” konusunu sabitler; bu konuda esneklik istenirse Command kalıbı kullanılır.

1.2 GoF örneği: menü öğeleri

Slayttaki örnekte uygulama (Application) vardır, uygulamada menü (Menu), menüde menü öğeleri (MenuItem) bulunur (ekrandan, 00:10:00). Belki menü içinde menü de olabilir; slayt bunu göstermez. Hoca, bir kat daha eklenip menü içinde alt menüler istenirse Composite ile Command'ın birlikte kullanılması gerektiğini söyledi ve bunu alıştırma olarak bıraktı.

Bir menü öğesine tıklanınca onunla ilişkili Command nesnesi çalışır. Command sınıfının bir Execute() metodu vardır. Command soyut bir sınıftır; Execute()'u override eden somut alt sınıflarda iş yürütülür. Slayttaki ilk olasılık "bir menü seçeneği ile (MenuItem nesnesi) bir dosya açma komutu ilişkilendirilebilir" biçimindedir:

```
Command            : Execute()  (soyut)
OpenCommand        : Execute(), AskUser()     -- Command'ın alt sınıfı
OpenCommand ----application----> Application : Add(Document)

OpenCommand.Execute() notu:
    name = AskUser()
    doc = new Document(name)
    application->Add(doc)
    doc->Open()
```

OpenCommand'ın erişim noktası Execute() metodudur. Sınıfta ayrıca kullanıcıya hangi dosyanın açılacağını soran AskUser() metodu vardır; Execute() bunu çağırır. Komutun üzerinde yürütüleceği uygulamayı (Application) görmek zorundadır.

1.3 Tasarım kararı: ilişki üst sınıfa mı, alt sınıfa mı?

Hoca bu şemadan bir tasarım kararı çıkardı. Daha genel bir çerçeve hazırlanıyorsa ve Command'ın alt sınıflarının çoğu Application'a işaretçi tutacaksa, Command'dan Application'a bir referans tutulursa bütün alt sınıflar bu işaretçiye sahip olur.

Başka bir alternatif kopyala-yapıştır işleridir. PasteCommand de Command'ın alt sınıfıdır; Execute() içinde uygulamanın o anda açık olan belgesi üstünde yapıştırma yapılır. O zaman Document sınıfına bir işaretçi gerekir. Ama OpenCommand'da belgeye bir işaretçi zaten yoktur. Daha özel bir çatı, örneğin bir ofis uygulaması yapılıyorsa belge ilişkisi üst sınıfa alınabilir. Genel bir GUI çatısı düşünülüyorsa Document ilişkisi yalnız PasteCommand'da kalmalı, üst sınıfa taşınmamalıdır.

2. Command Kalıp Yapısı

Slayt 20'deki kalıp yapısı (ekrandan, 00:12:20):

```
Client
Invoker  <>-------> Command : Execute()   (soyut)
                       ^
                       |
Receiver : Action() <---receiver--- ConcreteCommand : Execute(), state
                                      Execute() notu: receiver->Action();
Client -----> Receiver
Client - - -> ConcreteCommand   (kesikli: oluşturma)
```

Bileşenlerin adları kendini anlatır. Komutu veren, emri veren Invoker'dır. Komutun üstünde yürütülmesi gereken varlık Receiver'dır. Komutun kendisi, soyut Command sınıfının somut bir alt sınıfıyla (ConcreteCommand) modellenir. Çerçeveyi kullanacak Client da komut verebilecekse ona da bir ilişki çizilebilir; dosya menüsü gibi bir yapı söz konusuysa Invoker ile Command arasında parça-bütün ilişkisi olabilir.

Komutun bir alıcısı (receiver) vardır. Komutla ilgili ek durum bilgisi (state) de olabilir. Client, çeşitli Invoker'ları çeşitli ConcreteCommand'larla ilişkilendirme işini yapar. Slayttaki açıklamaya göre Client komut gerçekleme nesnesini (ConcreteCommand) oluşturur ve komutun alıcısını belirler; Client'tan ConcreteCommand'a çizilen kesikli çizgi bu oluşturma anlamındadır. Receiver'dan ConcreteCommand'a da bir ok çizilebileceği belirtilir.

Command'ın Java anlamında katı bir interface olması gerekmez. Önceki şemada her somut komutun bir receiver'ı olacağı düşünülmüştür. Ama alt sınıfların çoğu aynı command-receiver ilişkisini kullanacaksa bu ilişkiyi üst sınıfa taşımak yararlı olabilir. Hoca bu seçeneği akılda tutmayı önerdi.

3. Örnek Gerçekleme: Akıllı Ev Kumandası

Örnek Head First Design Patterns kitabından alınmıştır. Akıllı ev yazılımı üretiliyor. Merkezi bir kontrol paneli ve bu panelle ilişkilendirilebilecek çeşitli bileşenler var. Bunlar genelde bir şeyi açıp kapatan düğmelerdir: basit bir aydınlatma düğmesi olabileceği gibi iklimlendirme cihazları ya da kahve makinesi gibi elektronik aletler de olabilir.

Önerilen yapıda Command rolündeki arayüz Java interface'i olarak ele alınmıştır; örnekte kalıtılacak ortak davranış olmadığı için soyut sınıfa gerek yoktur. Açma ve kapama komutları vardır. Açılıp kapatılabilecek aletler aydınlatma düğmeleri, elektronik aletler ve iklimlendirme cihazlarıdır. Her birinin kendi constructor'ı vardır; hoca bu constructor'ların ortak bir üst sınıfa taşınabileceğini de söyledi. Ana program (kalıptaki Client) komut nesnelerine sahip olacak kumanda nesnesini kurar; ayrıca "eve gelince sokak kapısı açıldığında ne yapılacak", "evde kimse kalmadığında ne yapılacak" gibi akıllı işler için metotları vardır.

Kaynak kodlar (ekrandan, 00:20:00):

```
package dp.command.example;
public interface AcKapa {
    public boolean ac( );
    public boolean kapa( );
}

package dp.command.example;
public class AydinlatmaDugmesi implements AcKapa {
    private String isim;
    public AydinlatmaDugmesi(String isim) { this.isim = isim; }
    public String toString( ) { return "AcKapa:" + isim; }
    public boolean ac() { return true; }
    public boolean kapa() { return true; }
}
```

Constructor ismi alır; toString() bunun bir aç-kapa düğmesi olduğunu ve hangi alete verilen ad olduğunu döndürür. Gerçekte bir ev ağı üzerinden bir şeyi açıp kapatmanın ayrıntılı standartları vardır; hoca bunları bilmediği için iskelet kod olarak `true` döndürdüğünü söyledi. Diğer düğmeler de hemen hemen aynıdır.

Kumanda sınıfı (ekrandan, 00:21:00):

```
package dp.command.example;
import java.util.*;
public class Kumanda {
    private ArrayList<AcKapa> acKapaDugmeleri;
    public Kumanda( ) { acKapaDugmeleri = new ArrayList<AcKapa>(); }
    public void dugmeEkle( AcKapa dugme ) { acKapaDugmeleri.add(dugme); }
    public boolean ac( int dugmeNo ) {
        if( acKapaDugmeleri.get(dugmeNo) == null )
            return false;
        return acKapaDugmeleri.get(dugmeNo).ac();
    }
    public boolean kapa( int dugmeNo ) {
        if( acKapaDugmeleri.get(dugmeNo) == null )
            return false;
        return acKapaDugmeleri.get(dugmeNo).kapa();
    }
    public void hepsiniKapa( ) {
        for( AcKapa dugme : acKapaDugmeleri )
```

```
            dugme.kapa();
        }
}
```

Kumanda aç-kapa düğmelerini jenerik bir liste içinde tutar. Default constructor listeyi ilklendirir. dugmeEkle yalnız listeye ekler; örnek basit olsun diye aynı düğmenin iki kez eklenmesi gibi kontroller yapılmaz. Aç komutu belirli numaralı düğmeyi açacak biçimde kodlanmıştır: hocanın anlatımına göre o numarada düğme yoksa iş yapılamamıştır ve false dönülür, varsa düğmenin ac() metoduyla iş yaptırılır. Kapa da benzerdir. Açık ne varsa hepsini kapatmak yararlı olacağı için kumandaya hepsiniKapa da eklenmiştir.

> **Ders sonrası not:** `acKapaDugmeleri.get(dugmeNo) == null` sınaması, olmayan bir düğme numarasında false döndürmez. Java'da `ArrayList.get` geçersiz bir sırada null döndürmez, `IndexOutOfBoundsException` fırlatır (küçük bir Java programıyla denendi). Hocanın anlattığı davranış için sınamanın `dugmeNo < 0 || dugmeNo >= acKapaDugmeleri.size()` olması gerekir.

İşin içine biraz akıl katmak için ana uygulamada senaryolar tanımlanmıştır: ev boşken sokak kapısı açılınca önce antre ışığı açılır; eve gelen dinlenmek isteyeceği için televizyon da otomatik açılır; hava soğuksa kombi açılır. Hoca örneğin karışık olmadığını, Command uygulaması deneyimi olanların fikirlerini paylaşabileceğini söyledi.

4. Iterator Kalıbı: Amaç ve İki Çözüm

4.1 Amaç

Veri yapılarıyla çalışanlar iterator kavramını zaten bilir. Iterator kalıbının amacı, parçalardan oluşan bir bütünün parçalarını, bu parçaların iç yapısını açığa çıkarmak zorunda kalmadan, istenen herhangi bir biçimde dolaşabilmektir. Parçalardan oluşan bir bütün denince ilk akla gelen bir veri yapısıdır: liste, ağaç ya da en basiti dizi.

Sorun şudur: veri yapısının kullanım için sunduğu arayüzü farklı dolaşım türleriyle şişirmek istemiyoruz. Bağlı bir listede normalde baştan sona gidilir. Çift bağlı listeler herhangi bir noktadan geriye gitmeye de izin verir. Baş düğüm son düğümü de gösteriyorsa liste dairesel dolaşılabilir; ortadan başlayıp bir tur yapmak yeterli olabilir ya da tekrar dolaşmak istenebilir, bu da bir seçenek olmalıdır. Bu dolaşım türlerine ek olarak listede Integer, String ya da ilkel olan veya olmayan herhangi bir tür tutulabilir. Hem veri yapısının ana türü (liste mi, ağaç mı) hem tutulan nesnelerin türü için ayrı ayrı overload edilmiş ya da farklı adlı metotlar yazmak yerine daha akıllıca bir çözüm olarak Iterator kalıbı kullanılabilir.

Java'nın geniş bir koleksiyon çatısı (collection framework) vardır ve herhangi bir koleksiyon üzerinde iterator destekler. Hocaya göre Java bunu daha basit biçimde, kalıbın bütün özelliklerini kullanmadan gerçekler ve bu kötü bir şey değildir.

> **Hocanın vurgusu:** KISS ilkesi vardır: "Keep it simple, stupid." İşi basit tutmazsak aptalca bir iş yapmış oluruz. Java'nın hazır yapısı işimize yarıyorsa tabii ki onu kullanacağız (00:25:54).

4.2 Birinci çözüm

Bir List yapısı ve bir ListIterator sınıfı olsun; bir ListIterator bir List ile ilişkilidir. First() listenin ilk elemanına gider (current item listenin başı olur). Next() şu an gösterilen düğümden bir sonrakine geçer. CurrentItem() o anki elemanı döndürür. IsDone() dolaşımın bitip bitmediğini söyler. Bir liste iterator'a verildikten sonra bir döngüde "bitmediği sürece önce o anki elemanı al, sonra bir sonrakine geç" diye yazılabilir. List'te ise count, araya eleman ekleme, eleman silme, başa ekleme gibi bir liste veri yapısından beklenen her türlü metot olur.

Bu çözüm veri yapısıyla onun üstündeki dolaşma biçimini birbirinden soyutlar. Dezavantajı, List ile ListIterator'ın ilişkisidir: ağaç için ağaç iterator'ı, map için map iterator'ı, hash table için hash table iterator'ı diye uzar ve istemci somut türlere bağlanır.

4.3 İkinci çözüm

Bu zayıflığı gidermek için genel bir liste yapısı arayüzü ve genel bir dolaşım biçimi arayüzü oluşturulur; bunların somut alt sınıflarıyla belirli veri yapıları ve dolaşma biçimleri oluşturulur (ekrandan, 00:30:00):

```
Client ----> AbstractList : CreateIterator(), Count(), Append(Item), Remove(Item), ...
Client ----> Iterator     : First(), Next(), IsDone(), CurrentItem()

AbstractList'in alt sınıfları: List, SkipList
Iterator'ın alt sınıfları   : ListIterator, SkipListIterator

List     - - - (oluşturur) - - -> ListIterator      ve  ListIterator     ----> List
SkipList - - - (oluşturur) - - -> SkipListIterator  ve  SkipListIterator ----> SkipList
```

Listelerin bağlı liste, çift bağlı liste, dairesel liste gibi birçok çeşidi vardır; hepsinde dolaşabilecek bir Iterator arayüzü tanımlanır ve istemci işini bu soyut sınıflar ya da arayüzler üzerinden yürütür. SkipList, dolaşırken bazı elemanları atlayan, örneğin ikişer ikişer ya da üçer üçer ilerleyen bir yapı olabilir. Belirli bir iterator alt sınıfı belirli bir liste ile ilişkilidir: ListIterator List ile, SkipListIterator SkipList ile. İstemci, List ile mi SkipList ile mi çalıştığını bilmeden AbstractList arayüzü üzerinden istediğini yaptırır ve kendisine verilen somut iterator üzerinden hepsini eş biçimli olarak dolaşır.

4.4 Şema benzerliği uyarısı

Hoca bu yapının Bridge'e benzediğini söyledi ve genel bir uyarı yaptı.

> **Hocanın vurgusu:** Birbirinden farklı iki tasarım kalıbının sınıf şeması birbirine çok benzeyebilir. "Şeytan ayrıntıda gizlidir": onları ayıran, kalıpların çalışma biçimleri, amaçları ve gerçekleme ayrıntılarıdır. Birbirine benzeyen iki kalıbın farklarının ne olduğu sorulabilir (00:29:13, 00:29:36).

5. Iterator Kalıp Yapısı ve Gerçekleme Ayrıntıları

Kalıp yapısında bir Aggregate (birleşik, parçalardan oluşan bütün) ve onun parçaları arasında dolaşacak bir Iterator vardır. Bunlar soyut sınıf ya da arayüzdür; iş, somut gerçeklemeleri (ConcreteAggregate, ConcreteIterator) üzerinden yürütülür. Hoca, gerçekleme ayrıntılarının hepsinin düşünülüp ona göre tasarlanması gerektiğini söyledi.

- Iterator, üstünde gezdiği bileşenin durumunu değiştirebilmeli mi? Buna izin verilmemesi garanti edilmek isteniyorsa liste kendi klonunu oluşturup iterator'ın klonun elemanları üstünde dolaşmasını sağlayabilir; böylece asıl parçalar değiştirilemez. İzin verilecekse elemanlar değerle (call by value) değil referansla (call by reference) verilir ve düzenleme yapılabilir.
- Dolaşım algoritmasını kim gerçekleştirmeli: iterator mu, aggregate mi? Bu bir dolaşım biçimi olduğuna göre iterator'da kodlamak daha mantıklıdır.
- Iterator elemanları tek tek mi verecek, toplu mu? Iterator, aggregate'i dolaştıkça elemanları biriktirip hepsini toplu biçimde istemciye verebilir. Sondan başa gitmesi gerekiyorsa yine tek tek dolaşılır ama sonunda ters sırada istemciye verilir. Bu iş ConcreteAggregate'te de kodlanabilir, ama iterator'da daha mantıklı görünür.

  > **Hocanın vurgusu:** Bir sorumluluk, onu yerine getirmek için en fazla ya da en temel bilgi kimdeyse ona verilir (00:33:28).

6. Örnek Gerçekleme: İki Restoranın Menüsü

6.1 Problem

Örnek Head First Design Patterns'tan alınmıştır. Pandemi döneminde yemek siparişi uygulamaları çok yaygınlaştı; bu alanda bir fırsat gördük. En hızlı yol hazır iki firma almak: biri kahvaltılıkta, öteki ana yemeklerde uzmanlaşmış. İki firmanın kendi menü gerçeklemeleri var ama biraz farklı biçimde yazılmışlar. Bu menüler tek bir menü olarak müşteriye sunulacak.

Ayrıca yalnız vejetaryen yemekleri, yalnız kahvaltılıkları ya da yalnız ana yemekleri listeleme istekleri de olabilir. Firmalar yemekleri vejetaryen olarak işaretleyebilecek altyapıyı kurmamış olsaydı iş daha zor olurdu; senaryoda bunu hazırlamış oldukları varsayılır.

6.2 Firmaların menü sınıfları

İki firma da şans eseri ortak bir `MenuItem` sınıfı kullanmıştır; hoca bunu genel geçer bir çatı gibi düşünmeyi önerdi. Kullanmasalardı farklı yapıları tek bir iş etrafında toplamak için ek kod yazmak gerekecekti. MenuItem'da yemeğin adı, açıklaması, vejetaryen olup olmadığı ve fiyatı vardır.

Pancake House firması menü elemanlarını bir `ArrayList<MenuItem>` içinde tutar; diğer firma gibi dizi kullanmamış, daha yetenekli bir veri yapısı seçmiş ve Object listesi yerine jenerik tür kullanmıştır (ekrandan, 00:40:00):

```
ArrayList<MenuItem> menuItems;

public PancakeHouseMenu() {
    menuItems = new ArrayList<MenuItem>();
    addItem("K&B's Pancake Breakfast",
        "Pancakes with scrambled eggs, and toast", true, 2.99);
    addItem("Regular Pancake Breakfast",
        "Pancakes with fried eggs, sausage", false, 2.99);
    addItem("Blueberry Pancakes",
        "Pancakes made with fresh blueberries", true, 3.49);
    addItem("Waffles",
        "Waffles, with your choice of blueberries or strawberries", true, 3.59);
}
public String toString() { return "Pancake House Menu"; }
public void addItem(String name, String description,
                    boolean vegetarian, double price) {
    MenuItem menuItem = new MenuItem(name, description, vegetarian, price);
    menuItems.add(menuItem);
}
public ArrayList<MenuItem> getMenuItems() { return menuItems; }
```

toString() yalnız menünün adını döndürür; hoca bunun bütün menüyü dökecek biçimde de yazılabileceğini, ama zaten bir sonraki çözümde iterator ile dolaşılacağı için gerek görülmediğini söyledi. addItem bütün bilgileri parametre olarak alıp ekler; getMenuItems bütün menü seçeneklerini döndürür.

Diner firması biraz daha acemice davranıp dizi kullanmıştır. Diziye eleman eklemek daha zor olduğu için addItem'da dizide yer olup olmadığı gibi denetimler yapılır. Onun getMenuItems metodu ise dizi döndürür. Yani iki firmanın elemanları elde etme metotları farklı türler döndürür.

6.3 Kalıpsız çözüm: WaitressQuickAndDirty

Menüyü garson (Waitress) gösterir. Kalıpsız çözüm `dp.iterator.exampleWithoutPattern` paketindedir. Hoca "quick and dirty fix" tabirini hatırlattı: akla gelen ilk çözüm uygulanmıştır, çok iyi değildir ama çalışır. Garson iki menü nesnesini ilklendirir. Menü istendiğinde önce başlık basılır, sonra kahvaltılıklar ArrayList üzerinde, ana yemekler dizi üzerinde ayrı döngülerle yazılır (ekrandan, 00:43:40):

```
    MenuItem item = breakfastItems.get(i);
    printMenuItem(item);
}
System.out.println("\nLUNCH");
MenuItem[] dinerItems = dinerMenu.getMenuItems();
for( int i=0; i<dinerItems.length; i++ ) {
    MenuItem item = dinerItems[i];
    printMenuItem(item);
```

```
}
}
private void printMenuItem( MenuItem menuItem ) {
    System.out.print(menuItem.getName() + ", ");
    System.out.print(menuItem.getPrice() + " -- ");
    System.out.println(menuItem.getDescription());
}

public void printVegetarianMenu() {
    System.out.println("\nVEGETARIAN MENU\n----\nBREAKFAST");
    ArrayList<MenuItem> breakfastItems = pancakeHouseMenu.getMenuItems();
    for( int i=0; i<breakfastItems.size(); i++ ) {
        MenuItem item = breakfastItems.get(i);
        ...
```

Bir menü öğesinin adını, fiyatını ve açıklamasını yazan private `printMenuItem` metodu sayesinde biraz metotlaştırma yapılmış ve tekrar azaltılmıştır. Ama iki for döngüsü arasında yine kopyala-yapıştır ilişkisi görülür. Duplicate code'un "ağababası" ise `printVegetarianMenu`'dadır: printMenu ile neredeyse aynıdır, yalnız her menü öğesinin vejetaryen olup olmadığına bakıp öyleyse yazar. Yalnız kahvaltı ve yalnız ana yemek basan metotlar da vardır; örneğin printBreakfastMenu ile printMenu'nun ilk yarısı aynıdır. Bu çözümde garson iki menü sınıfıyla doğrudan ilişkilidir.

6.4 Iterator kalıbı ile çözüm

Kalıplı çözümün sınıf şeması (ekrandan, 00:50:00):

```
<<interface>> Menu : + createIterator() : Iterator<MenuItem>
<<interface>> Iterator<MenuItem> : + hasNext() : boolean, + next() : MenuItem

Waitress ----> Menu
Waitress - - > Iterator<MenuItem>
DinerMenu, PancakeHouseMenu  - - -|> Menu                (gerçekler)
DinerMenuIterator, PancakeHouseMenuIterator - - -|> Iterator<MenuItem>
DinerMenu <-----> DinerMenuIterator
PancakeHouseMenu <-----> PancakeHouseMenuIterator
```

İlk bakışta daha karmaşık görünür: farklı iterator'lar vardır, menüler ve iterator'lar ortak arayüzler gerçekler, yani bileşen sayısı artmıştır. Ama sadelik ve esneklik açısından kazanç olup olmadığına bakmak gerekir.

Firmaların menü sınıflarında neredeyse hiçbir şey değişmez. Hoca iki sürümü Eclipse'te ikisini seçip sağ tıklayıp "Compare With" ile karşılaştırarak gösterdi: yalnız ortak `Menu` arayüzünü gerçekleme bildirimi ve `createIterator()` metodu eklenmiştir. DinerMenu'nun createIterator()'ı bir DinerMenuIterator oluşturur ve ona kendi elemanlarını dizi olarak verir; PancakeHouseMenu ise bir PancakeHouseMenuIterator oluşturup ona kendi menü öğelerini verir. Mevcut koda yalnız iki küçük ekleme yapılmıştır.

DinerMenuIterator (ekrandan, 00:48:00):

```
package dp.iterator.exampleWithPattern;
import java.util.*;
public class DinerMenuIterator implements Iterator<MenuItem>{
    private MenuItem[] list;
    private int position = 0;

    public DinerMenuIterator(MenuItem[] list) {
        this.list = list;
    }

    public MenuItem next() {
        MenuItem menuItem = list[position];
        position = position + 1;
```

```
        return menuItem;
    }

    public boolean hasNext() {
        if (position >= list.length || list[position] == null) {
            return false;
        } else {
            return true;
        }
        ...
```

Constructor'a üstünde dolaşacağı dizi verilir. next() o anki konumdaki elemanı alır, konumu bir ilerletir ve elemanı döndürür. hasNext() konum dizinin sonuna ulaştıysa ya da o konumdaki eleman null ise false, aksi halde true döner. remove() ise "çabuk sahaya çıkmak istedik" diye sonraya bırakılmıştır. PancakeHouseMenuIterator'ın işi daha kolaydır, çünkü ArrayList'in kendi metotları vardır; yapısı benzerdir.

Waitress sınıfı (ekrandan, 00:50:40):

```
    this.pancakeHouseMenu = pancakeHouseMenu;
    this.dinerMenu = dinerMenu;
}

public void printMenu() {
    Iterator<MenuItem> pancakeIterator = pancakeHouseMenu.createIterator();
    System.out.println("MENU\n----\nBREAKFAST");
    printMenu(pancakeIterator);
    Iterator<MenuItem> dinerIterator = dinerMenu.createIterator();
    System.out.println("\nLUNCH");
    printMenu(dinerIterator);
}

private void printMenu(Iterator<MenuItem> iterator) {
    while (iterator.hasNext()) {
        MenuItem menuItem = iterator.next();
        printMenuItem(menuItem);
    }
}
private void printMenuItem( MenuItem menuItem ) {
    System.out.print(menuItem.getName() + ", ");
    ...
```

İki menü aynı ortak Menu arayüzünü desteklediği için garson onları aynı türden değerlendirebilir; arka planda iki farklı veri yapısı olduğunu ve her birinde farklı dolaşmak gerektiğini bilmek zorunda değildir. printMenu çok sadeleşmiştir: Pancake House menüsünden bir iterator alınır, başlık yazılır ve basılır; sonra Diner menüsünden iterator alınıp ana yemekler basılır. "Iterator'da daha fazla eleman olduğu sürece sıradakine git ve yazdır" döngüsü tek bir yerde kodludur. Kalıpsız çözümde ise bu dolaşma kodu farklı yerlerde tekrar ediyordu.

Vejetaryen menü de ayrı bir metotla, iterator verilerek basılır ve yine tek yerde kodludur. printMenu ile printVegetarianMenu arasında hâlâ biraz benzerlik vardır. Hoca, istenirse printMenu'ya ikinci bir parametre (yalnız vejetaryen mi) eklenip true ise yalnız vejetaryen öğelerin basılmasıyla bu tekrarın da tek noktada toplanabileceğini, örnekte bunun tercih edilmediğini söyledi. Yalnız kahvaltı basılacaksa Pancake House menüsünün iterator'ı verilerek alt metot çağrılır.

6.5 Bu çabaya değer mi?

Hocaya göre değer. İş büyüdükçe tatlı menüsü, Çin mutfağı, Akdeniz mutfağı gibi yeni menüler gelecektir. Kalıplı yapıda yeni bir menü geldiğinde yalnız o menü sınıfı ve onda nasıl dolaşılacağını bilen iterator'ı eklenir. Kalıpsız çözümde ise sınıf ilişkileri ve kopya kod artar. İki menü varken iki çözüm arasında bir sınıf sayısı farkı vardır, ama menü sayısı arttıkça bu

fark göreli olarak azalır. Kalıplı çözümle daha az kopyala-yapıştır kod yazılır ve işe odaklanılır.

7. Memento Kalıbı

7.1 Amaç

Memento "anı, hatıra" demektir. Hoca aynı adlı filmi, kısa dönem hafızasını yitirmiş bir kahramanı anlatan eski ama tavsiye ettiği bir film olarak andı.

Slayttaki amaç maddeleri (ekrandan, 01:10:00):

- Bir nesnenin durum bilgisini sarmalama (encapsulation) ilkesini bozmadan elde etmek.
- Böylece nesnelerin ikincil saklama ortamlarında saklanması, geri yüklenmesi, ağ üzerinden aktarımı mümkün olabilir.
- Yine bu şekilde sona yapılan işlemi geri alma düzeneği (undo) kurulabilir.
- Durum bilgisinin sadece bir kısmının saklanması da yeterli olabilir.

Durum bilgisi belki diske bir dosyaya yazılacak, belki ağ üzerinden byte olarak gönderilecek, belki nesnenin zaman içindeki değişikliklerinin geçmişi tutulup bir önceki ya da bir sonraki haline dönülecektir. Bunu yaparken nesnenin durumu bir bütün olarak, üstünde oynanmadan saklanmak istenir. Nesnenin kendisi saklansaydı public setter metotları üzerinden durumu değişebilirdi; oysa saklandığı andaki hali bozulmadan korunmak istenir. Nesnenin beş üye alanından yalnız bazıları ilgileniyorsa yalnız onlar bir memento altında toplanabilir.

7.2 Kalıp yapısı

Slayt 39 (ekrandan, 01:12:10):

```
Originator : SetMemento(Memento m), CreateMemento(), state
Memento    : GetState(), SetState(), state
Caretaker <>---memento---> Memento
Originator - - -> Memento   (oluşturur)

CreateMemento() notu:  return new Memento(state)
SetMemento(m) notu:    state = m->GetState()
```

Memento, durum bilgisi saklanmak istenen içeriktir. Mementoyu oluşturan Originator'dır; durumu izlenen nesne de odur. Originator'ın belli bir andaki durumu bir memento nesnesiyle saklanır.

Hocanın benzetmesi: çocuğunuzun gelişimini her yıl çektirdiğiniz fotoğraflarla saklamak istiyorsunuz. Çocuk Originator'dır, çekilen resimler Memento'dur, fotoğrafları saklayan Caretaker'dır. Fotoğrafların dışarıdan biri tarafından değiştirilmediğinden emin olmak gerekir. Bu yüzden Memento'daki durumun dışarıdan set edilememesi istenir, ama Originator onu set edebilmelidir. Caretaker memento'ların listesini tutar ve belli bir anda Originator'a "bu mementoyu senden almıştım, al, durumunu buna döndür" diye geri verir; bunu yaparken içeriği değiştirememelidir. Kalıbın asıl anlamı budur: set metodunu yalnız Originator çalıştırabilsin, dışarıdan oynanmasın.

Originator anlık durum bilgisine sahiptir. Memento, durum bilgisinin geçerli halini saklar. Caretaker durum bilgisinin kopyalarını tutar ve SetMemento ile bu kopyalardan biri geçerli durum haline getirilebilir.

Kalıp şu durumda kullanılır: nesnenin durum bilgisinin tümünün ya da bir kısmının, daha sonra o duruma geri dönmek üzere saklanması gerektiğinde ve nesneye doğrudan bir işaretçi verilmek istenmediğinde. İkinci madde önemlidir, çünkü öteki seçenek nesnenin kendisini saklamaktır.

7.3 Dillerde erişim denetimi sorunu

Bu istek her dilde desteklenmez. Nesneye dayalı yaklaşımın ana mantığında public ve private vardır: public'e herkes erişir, private'a yalnız aynı türden nesne erişir. Bazı dillerin ek mekanizmaları vardır ama bunlar her dilde geçerli değildir.

- C++'ta iş daha kolaydır: Memento'nun bütün metotları private yapılır ve Originator sınıfı friend olarak tanımlanır. Bir sınıf başka bir sınıfı friend ilan ederse private üyeleri onunla paylaşılır.
- Java'da friend yoktur. Ama paket kuralı vardır: Memento ile Originator aynı pakete, Caretaker başka bir pakete konursa sorun çözülebilir. Bu durumda paketleri mühürlemeyi (seal) unutmamak gerekir; yoksa biri o jar dosyasında aynı pakete herhangi bir sınıf koyup paket düzeyindeki bütün üyelere erişebilir.
- İç sınıf (inner class) mantığıyla da çözülebilir. Hoca, iç sınıf yapısının C#'ta da olduğunu düşündüğünü söyledi.

Özellikle Java'da şu durum vardır: CreateMemento metodu durumu doğrudan göndermemelidir. Durumu oluşturan içeriğin klonu yapılmalı ve CreateMemento bu klonu döndürmelidir; aksi halde yine erişim sorunları olabilir.

> **Ek bilgi (derste anlatılmadı):** Java'da bir nesne başka bir nesneye verildiğinde kopyası değil referansı geçer. Memento içinde durum nesnesinin referansı tutulursa Originator bu nesneyi sonradan değiştirdiğinde memento'daki "eski" durum da değişmiş olur. Klonlama, memento'nun o anki değerlerin bağımsız bir kopyasını tutmasını sağlar.

Örnek olarak bir belge düşünülebilir: belgenin memento'ları tutulur ve undo-redo yapılmak istenir. Herhangi bir işlem yapılmadan önce durum bilgisi eklenir, sonra işlem gerçekleştirilir. Undo denince bir önceki memento, redo denince bir sonraki memento yüklenir. Hoca bu örneğin kodunu öğrencilerin de yazabileceğini söyledi.

8. Örnek Gerçekleme: İç Sınıf ile Memento

Hoca, kendisine zamanında zaman kazandıran iç sınıf çözümünü gösterdi (`dp.mementoV2.innerClass`, istemci `dp.mementoV2Outer`).

- `State`: Originator'ın durumu ayrı bir sınıf olarak tutulur. Klonlanabilir (Cloneable) yapılmıştır. Durum bilgisi olarak bir String içerik düşünülmüştür; getter ve setter'ları vardır. Kendi klonlama metodu, kendinin bir kopyasını oluşturup durum bilgisini de verdikten sonra kopyayı döndürür.
- `Originator`: bir State üyesi vardır, constructor'da ilklendirilir. Herhangi bir anda `createMemento()` ile bir memento alınabilir; kendinden yeni memento istendiğinde kendi durumunu memento'ya verir. Memento setter metodu (`setMemento`) vardır. Durumu değiştirmek için `modify` metodu vardır. `toString()` durum bilgisini döndürür.
- `Memento`: Originator'ın public iç sınıfıdır. Durumu vardır, getState ve setState metotları vardır, ama bu metotlar private'tır. Sınıfın kendisi private yapılsaydı dışarıdan hiç kullanılamaz ve anlamı kalmazdı. Diğer her şey private olduğu halde Memento, Originator'ın iç sınıfı olduğu için Originator ona her şekilde erişebilir.

Hoca, alınmış bir memento'nun durumunun Caretaker'da değiştirilip "al, bu duruma dön" denmesini istemediğini, Originator'ın "bunu ben oluşturmuştum, sen bozmuşsun, kabul etmiyorum" diyebilmesi gerektiğini söyledi.

Caretaker'daki normal kullanım (ekrandan, 01:30:00):

```
package dp.mementoV2Outer;
import dp.mementoV2.innerClass.*;
import dp.mementoV2.innerClass.Originator.Memento;

public class Caretaker {
    public void test( ) {
        Originator subject = new Originator( );
        subject.modify("ilk durum");
        Memento mem = subject.createMemento( );
        subject.modify("yeni durum");
        //bir süre sonra...
        subject.setMemento( mem );
        System.out.println(subject);
    }
```

```
    public static void main( String[] args ) {
        Caretaker ct = new Caretaker();
        ct.test();
    }
}
```

Bir Originator oluşturulur, durumu “ilk durum” yapılır, bir memento alınır, durum “yeni durum” yapılır. Geri dönmek istenince ilk durumdayken alınan memento verilir ve yazdırılır. Çalıştırınca nesnenin ilk duruma döndüğü görülür.

Hoca klonlamanın neden gerektiğini kendi deneyimiyle anlattı: State’i klonlamadan döndürdüğünde nesne referansları paylaşılmaya devam ediyordu; eski memento’yu verse de Originator yazdırılınca hâlâ “yeni durum”u gösteriyordu. Araştırınca durumun Cloneable olup klonunun verilmesi gerektiğini bulmuştu.

Caretaker, `mem` üzerinden memento’nun durumunu değiştirmeye çalışırsa erişemez: Memento public bir iç sınıf olsa da üyeleri private olduğu için Caretaker onu tutabilir ama değiştiremez.

Kırma denemesi `testBroken()` (ekrandan, 01:20:00):

```
public void testBroken( ) {
    Originator subject = new Originator( );
    subject.modify("ilk durum");
    subject.modify("yeni durum");
    //bir süre sonra...
    Originator fakeSubject = new Originator( );
    fakeSubject.modify("sahte durum");
    Memento memFake = fakeSubject.createMemento();
    ...   // subject.setMemento( memFake ) ve yazdırma (slayttaki açıklamaya göre)
}
```

Burada gerçek nesneye, başka (sahte) bir Originator’dan alınmış bir memento verilir. Hoca bu çözümün bu durumu engelleyemediğini hatırladığını söyledi (“hırsıza kilit dayanmaz” deyişini kullandı). Slayttaki açıklamada (2020’den kalma notlar) şu çözüm önerileri vardır (ekrandan, 01:30:00):

- Memento’ların idaresi Caretaker’a değil Originator’ın kendisine verilirse, Caretaker da setMemento yerine `activateMemento(int siraNo)` çağırırsa sorun çözülür. Ancak illa harici bir Caretaker bakacaksa böyle bir çözüm düşülmüştür:
- Originator bir memento oluştururken kendi hashCode’unu ya da kurucuda rastgele atanan bir seri numarasını memento’ya verir ve uyumsuz kodlu memento’yu almayı reddeder. hashCode yalnız program çalıştığı sürece geçerlidir; programı kapatıp açınca her şey geçersiz olur. Rastgele seri numarası daha kalıcı bir çözüm üretmede kullanılabilir.

Hoca, memento’ların idaresini Originator’ın yapması yeterli görülürse bu çözümün gayet yeterli olduğunu, harici Caretaker isteniyorsa çok ağır olmayan bir çözüm bulan öğrencilerin önerilerini beklediğini söyledi.

9. Observer Kalıbı

9.1 Amaç ve örnek

Observer “gözleyici, gözetleyici” demektir. Bir nesne vardır ve durumu yakından izlenmek istenir. O nesnenin durumu değiştiğinde onunla ilişkili bütün diğer nesneler durumdan haberdar olmalı ve güncel bilgiye erişebilmelidir. Gözlenen bir nesne ve farklı türlerden olabilen gözleyicileri vardır. Bu ilişkileri sınıf adları üzerinden doğrudan bağlaşım kurmadan, çalışma anında kuracak bir çözüm aranır.

Slayttaki örnek Model-View-Controller altyapısına benzer: hesap tablosundaki veri (model, subject) değiştiğinde tablo görünümü, çubuk grafik ve pasta grafik otomatik güncellenmelidir. Gözlenen nesne gözleyiciye “ben değiştim” der, gözleyici de haberdar olup gereğini yapar. Burada da “neyin değiştiğini bul ve sarmala” ilkesi işler.

9.2 Kalıp yapısı

Slayt 47 (ekrandan, 01:33:50):

```
Subject ----observers----> Observer : Update()   (bire-çok)
Subject : Attach(Observer), Detach(Observer), Notify()
    Notify() notu: for all o in observers { o->Update() }

ConcreteSubject (Subject'in alt sınıfı) : GetState(), SetState(), subjectState
    GetState() notu: return subjectState
ConcreteObserver (Observer'ın alt sınıfı) : Update(), observerState
    ConcreteObserver ----subject----> ConcreteSubject
    Update() notu: observerState = subject->GetState()
```

Gözlenen Subject sınıfından, gözleyiciler Observer sınıfından olur. Subject ile Observer arasında bire-çok ilişki vardır. Dörtlü çete bunu eski bir gösterimle, ok ucunda siyah bir nokta ile çizmiştir; standart UML'de çokluk 1 ve * olarak yazılır.

> **Hocanın vurgusu:** Dörtlü çetenin gösterimi UML önerilmeden önceki eski bir gösterimdir. Kalıtım ya da arayüz gerçekleme oklarının standart UML'e göre çizilmesini bekliyorum (01:33:12).

Subject'e bir gözleyici eklemek için Attach, artık gözlenmesi gerekmeyen gözleyiciyi çıkarmak için Detach metodu vardır. Kalıba özünü kazandıran Notify metodudur: Subject'te bir değişiklik olduğunda bütün gözleyicilerin Update metodunun çağrılmasını sağlar.

Subject'in somut alt sınıfları ConcreteSubject, Observer'ın somut alt sınıfları ConcreteObserver'dır. ConcreteObserver'ın ConcreteSubject'i bilmesi gerekir. Bazı gözleyiciler ConcreteSubject'in durumunun tümüyle değil, memento gibi bir alt kümesiyle ilgilenebilir. Her halükarda belirli bir ConcreteObserver'ın belirli bir ConcreteSubject'e erişebilmesi gerekecektir. Gözlenebilecek çeşitli varlıklar olacaksa Subject soyut yapılmayabilir. Belirli türden varlıklarla ilgilenen belirli türden gözleyiciler varsa alttaki ilişki kurulur; böyle bir durum yoksa ilişkiyi üste taşımak ya da çift yönlü yapmak yararlı olabilir. Bir observer'ın tek bir subject'i gözlemesi diye bir kural da yoktur; birden fazla subject'i gözleyebilir, o zaman ilişkinin o tarafına da çokluk konur. Bu, kalıbın en temel önerisidir ve ihtiyaca göre ayrıntılandırılabilir.

10. Observer Gerçekleme Ayrıntıları: Pull ve Push Modelleri

10.1 Durum bilgisinin gözleyicilere gönderilmesi

> **Hocanın vurgusu:** Observer kalıbında en önemli iş, Subject'in durum bilgisinin Observer'lara gönderilmesidir. Bunun iki farklı yolla yapılabileceğini iyi bilmeniz gerekir: pull model ve push model. İkisi birbirinin simetriğidir; birinin avantajı ötekinin dezavantajıdır (01:36:48).

Slayt 49 (ekrandan, 01:40:00):

- Pull model:
  - (+) Gözlenen sadece gözleyicileri haberdar eder, gözleyiciler ilgili durum bilgisini gözlenenden alır.
  - (+) Gözlenenin gözleyicilerle bağlantısını azaltan çalışma biçimidir.
  - (+) Gözleyici durum bilgisinin sadece istediği kısmını almayı seçebilir.
  - (−) Gözleyici gözlenende nelerin değiştiğini kendisi çıkarmak zorunda kalır.
- Push model: Gözlenen gözleyicilere durum değişikliği hakkında ayrıntılı bilgi gönderir.
- Aynı türden gözlenen nesnelerin tümünün gözleyicilerinin ortak olması istenirse, gözlenen gözleyicilerini statik bir veri yapısı içinde tutabilir.

Pull modelde gözleyici "daha önce böyle biliyordum, şimdi bu geldi, ne değişmiş?" sorusunu kendisi çözmek zorundadır; bu o tarafa ek yüktür. Push modelde subject belki kendi durumunun tümünü gönderir ya da "sıcaklık bilgim değişti", "şu bilgim değişti" gibi farklı metotlar olabilir. O zaman subject hangi tür gözleyicinin hangi metodunu çağıracağını bilmelidir ve subject'e daha fazla kod yazmak gerekir.

10.2 İletişimin pahalı olduğu ortamlar

Subject ile Observer farklı yerlerdeyse, örneğin IoT uygulamalarında, pull ya da push seçimine çok dikkat etmek gerekir. İletişim her durumda masraflıdır; hele kablosuz ya da GSM iletişimi ise ve hem yükleme hem indirme ayrı ücretlendiriliyorsa. Subject yalnız "değiştim" diyorsa o yöne küçük bir paket gider. Observer durumun yalnız bir alt kümesiyle ilgileniyorsa yalnız o veriler için getter çağırır ve subject'ten observer'a daha az durum bilgisi gider. Subject bütün durumunu yollarsa belki o kadar bilgiye gerek yoktur; daha fazla paket ve daha fazla haberleşme masrafı çıkar.

Ters yönden bakınca update'in çağrılması önemlidir. Observer'ın sürekli "değiştin mi, değiştin mi" diye sorması mantıksızdır: hem haberleşme masrafını hem boşuna harcanan işlemci gücünü artırır. Kalıbın en önemli noktası, subject'in "ben değiştim" demesidir.

10.3 Farklı türden gözleyiciler

Bir gözleyici birden fazla subject'i gözleyebilir; gözlenen nesnelerin farklı türden gözleyicilerce izlenmesi de istenebilir ve kalıp bunu destekler. Belirli bir ConcreteSubject'ten belirli bir ConcreteObserver'a bir ok olabilir. Üst sınıftan gelen attach metodunda gelen observer'ın gerçekten ilgilenilen türden bir ConcreteObserver olup olmadığı `instanceof` ile denetlenebilir; o zaman bir tür dönüşümü (typecasting) gerekebilir. Hoca typecasting'in pek güzel bir şey olmadığını, ama büyük resmi bozmadan küçük eklemelerin kalıba eklenebildiğini söyledi. Java'da GUI uygulamalarının olay mimarisi bu biçimdedir: GUI bileşenleri olay çıkarır ve bunlar için özel listener türleri vardır; örneğin klavye olaylarını bir keyboard listener dinler.

10.4 Haber vermeyi kim tetiklemeli?

Haber verme mesajını istemci tetiklerse, subject'i kullanan kişi bu komutu vermeyi unutabilir. Durumu değiştiren subject tetiklerse sorun olmaz, ama bu kez çok küçük değişikliklerin hemen bildirilmesi gerekmeyebilir. Hocanın örneği bir saksıdaki nem sensörüdür: değer 10.2'ye düşer, bir dakika sonra 10.1'e iner, biraz güneş gelir, yarım saat sonra bulut gelir ve 10.3'e çıkar. Sensör aşırı hassassa ve her küçük değişiklikte gözleyicilere mesaj giderse, pull ya da push olsun, sürekli veri akışı olur ve iletişim masrafı artar.

Bunun için belki gözleyiciler subject'e "değiştin mi" diye sorar; değiştiyse durum, değişmediyse null döner. Ama o zaman subject'in "bu ID'li gözleyici beni en son şu zamanda sordu, o zamandan beri değiştim mi" bilgisini tutması için ek kod gerekir. Hoca, haberleşme masrafının mı, işlemci gücünün mü yoksa daha karmaşık kod yazma çabasının mı önemli olduğuna bakılarak karar verileceğini söyledi.

10.5 Ne zaman kullanılır?

- Bir soyutlamanın birbirine bağımlı birden fazla parçası olduğunda. Örneğin bir composite'in bileşenlerinden birinde değişiklik olunca diğerlerinin de haberdar edilmesi istenebilir. Kök düğüm observer, ara ve yaprak düğümler subject olarak düşünülebilir; bir değişiklik köke push edilir, kök de composite'teki diğer bileşenlere bunu iletir.
- Bağlaşımı artırmadan bir nesnenin diğer nesneleri bir olgudan, durumdan ya da değişiklikten haberdar etmesi gerektiğinde.

11. Örnek Gerçekleme: Müşteri Kaydında Gözleyiciler

11.1 Çözüm 1

Bir müşteri ilişkileri programı vardır. Yeni bir müşteri kaydedildiğinde ona hoş geldiniz epostası gönderilsin ve müşterinin doğru bir ikamet adresi girip girmediği doğrulansın istenir. Müşteri yönetimiyle ilgili bir Customers sınıfının add metodu çağrıldığında bir WelcomeLetter ve bir AddressVerification işlemi yapılır. Bunların public static metotları olabilir, bu yüzden kullanım ilişkisi olarak verilmiştir; dışarıdan bir factory ile de verilebilirler.

11.2 Yeni gereksinim ve Çözüm 2

Slayt 52 (ekrandan, 01:50:00):

- Yeni bir gereksinim ortaya çıkıyor: müşteri bir şubemizin 10 km yakınında ise ona gönderilecek mektupta indirim kuponları da olsun.
- Benzer başka gereksinimlerin de ortaya çıkabileceğini görüp şimdiden önlemimizi alalım.
- Sisteme her yeni müşteri eklendiğinde bir veya birkaç farklı işlem yapmak üzere bir altyapı oluşturalım.
- Müşteri eklendiği zaman sistem yapması gereken işler olabileceğinden haberdar edilsin.
- Bu düşünce biçimi bizi observer kalıbına götürür.
- Brick-and-mortar: ağırlıklı olarak e-ticaret yapanlar, fiziksel binalarda yaptıkları ticareti böyle adlandırır.

Pazarlama departmanının böyle değişik fikirlerle gelmeye devam edeceği anlaşılınca daha esnek bir altyapı kurmaya karar verilir. Customer sınıfına çeşitli observer'lar eklenir; her biri bir işi yapmakla yükümlüdür.

Slayt 53 (ekrandan, 01:52:40):

```
Customer
  - myObs : Vector<ICObserver> {static}
  + attach(ICObserver) {static}
  + detach(ICObserver) {static}
  + getState()
  + notifyObs()        -- not: notifyObs metodu gözleyicilerin update
                          metodunu çağırarak onları haberdar eder.
Customer ----> <<interface>> ICObserver : + update(myCust : Customer)
ICObserver'ı gerçekleyenler: AddrVerification, BrickAndMortar, WelcomeLetter
```

Slayttaki son madde: sadece müşteri nesneleri izlenecekse bir soyut "Subject" üst sınıfı hazırlamaya gerek kalmadan observer kalıbı kullanılabilir.

Gözleyici arayüzüne bir update metodu konmuştur ve ana iş mantığı update'ten başlatılır. AddrVerification ve WelcomeLetter'ın kendi iş metotları (hocanın deyişiyle X, Y, Z metotları) olabilir; geriye doğru uyumluluk için bunların adlarını değiştirmek gerekmez, yalnız update metodu eklenir ve asıl işi başlatan metot ondan çağrılır. Bu çok yük getirmez. Ek olarak BrickAndMortar sınıfı, hoş geldiniz mektubuna indirim kuponunu ekleme işini kendi içinde yapar.

Customer'a üç somut gözleyici sınıfından nesneler attach edilir. Çok fazla kupon dağıtıldı ve "işler kesatlaşana kadar kupon vermeyelim" denirse BrickAndMortar gözleyicisi detach edilir; yeniden kupon verilmeye başlanınca tekrar attach edilir. Gelen istekler çok rahat karşılanır. notifyObs bütün gözleyicileri çağırır, "ben güncellendim, yeni gelen müşteri de budur" diye update metoduna müşteriyi verir; gözleyiciler de gereğini yapar. Ayrıntılı iskelet kod kaynak kodlarda mevcuttur.

12. Strategy Kalıbı

12.1 Amaç ve GoF örneği

Kalıbın amacı, aynı işe yönelik ama farklı avantaj ve dezavantajları olan algoritmalar ile bu işin yapılmasını isteyecek istemcileri birbirinden soyutlamaktır. Böylece kullanılacak algoritma çalışma anında değiştirilebilir. Algoritmaların avantaj ve dezavantajları bilindiğine göre o anki duruma bakıp "şu an şu biçimde çalışmak istiyorum" denebilir.

GoF örneği: metin ve kelime işleme yazılımlarında alt satıra geçme (line break) algoritmaları çok çeşitli olabilir. Belirli bir anda istenen algoritmayla çalışacak biçimde ayar yapılmak istenir.

Hoca bunu Word'de gösterdi: üç paragraflık örnek bir metinde, satıra sığmayan bir sözcük normalde bütün olarak alt satıra geçirilir. Paragraf "iki yana yasla" yapılınca program sözcükler arasındaki boşlukları gözle görülmeyecek kadar daraltarak sözcüğü yukarı taşıyabildi. Hece bölme istenebilir: sözcüğün bir kısmı satıra sığsın, bir kısmı alta geçsin (tek harfli hecenin bölünmemesi gibi dilbilgisi kuralları da vardır). Ya da hecelerin doğruluğuna bakmadan,

satır sınırı neredeyse oradan bölünsün denebilir; hocaya göre Word bu üçüncü seçeneği sunmuyordu ama eklenebilirdi. Her paragrafa ayrı ayrı bir yöntem seçilebilmeli ya da gelişmiş ayarlardan varsayılan davranış belirlenebilmelidir.

"Neyin değiştiğini bul ve sarmala" ilkesine göre burada değişen satır bölümleme algoritmalarıdır. Kesmeli bölme, kelimeden bölme ve sınırdan bölme yapan üç algoritma kodlanabilir; uygulama da bundan haberdar olmalı ve istenen anda "artık bu biçimde metin düzenleyeceksin" denebilmelidir. Bu yüzden SatırBölümleme diye soyut bir sınıf ve içine bölmele adlı soyut bir metot tanımlanır (ekrandan, 02:10:00):

```
Uygulama
  - bölmeleyici : SatırBölümleme
  + metinDüzenle()          -- not: bölmeleyici.bölmele()
Uygulama ----> <<abstract>> SatırBölümleme : + bölmele() {abstract}
SatırBölümleme'nin alt sınıfları: KesmeliBöl, KelimedenBöl, SınırdanBöl
                                  (her biri + bölmele())
```

Slayttaki not: varsayılan bir bölümleme algoritması söz konusu ise üst sınıfın ve ilgili metodunun soyut olması zorunlu değildir, ancak kalıbın genel yapısı bu şekildedir.

Uygulama metin düzenlerken bölmeleme ihtiyacı olunca ilişkili olduğu nesneye "bölmele" der. Bu soyut metot, uygulamaya hangi somut bölmeleyici verilmişse onun bölmele metodunu çalıştırır. Şemada gösterilmese de bölmeleyici üyesi için bir setter da olur: "artık kesmeli bölmeyle çalış" denince setter'la bir KesmeliBöl nesnesi verilir. Hoca, dörtlü çetenin örnek problemlerini Türkçeye çevirip düzgün UML şemasına koyduğunu belirtti.

12.2 Kalıp yapısı

Kalıp yapısında bir Context ve bu context'e uygulanabilecek çeşitli stratejiler vardır. Bir Strategy arayüzü ya da gerekiyorsa soyut sınıfı konur; algoritmanın gerektirdiği adımlar bu arayüzde metot olarak tanımlanır. Somut alt sınıflar (ConcreteStrategy) bu adımları A algoritmasına, B algoritmasına göre yürütecek biçimde kodlanır. Context'te bir setStrategy metodu olur; hangi strateji gerçeklemesi verilmişse context artık o algoritmayı kullanır.

Strateji ile context, seçilen algoritmayı gerçekleştirmek için etkileşir. Context kendi durum bilgisini stratejilere parametre olarak aktarabilir. Ya da stratejiden context'e bir ilişki kurulup stratejiler context'ten haberdar edilir. Bütün somut stratejilerin context'ten haberdar olması gerekiyorsa bu ilişki üst sınıfta kurulur; o zaman strateji bir interface olamaz, context türünden bir üye alanı olan soyut sınıf olur. Yalnız bazılarının ihtiyacı varsa ilişki yalnız onlarda kodlanır.

12.3 Ne zaman kullanılır, zayıf yönleri

Bir algoritmanın değişik çeşitlemelerine ihtiyaç duyulduğunda ve istemcinin kullanılacak algoritmanın gerçekleme ayrıntılarını bilmesi gerekmediğinde kullanılır.

Zayıf yön: istemciler ister istemez değişik stratejilerin varlığından haberdar olmak zorundadır. İstemci durumu değerlendirip "şu an şu algoritmayla çalışmalıyım" diyecekse somut strateji sınıflarını bilmesi gerekir. Bu işle ilgili üçüncü bir nesne varsa, durumu inceleyip uygun stratejiyi seçen ve istemciye veren o olur; istemci de verilen stratejiyle çalışır.

Stratejiden context'e olan ilişki, hele context ve algoritmalar farklı sunucularda barındırılıyorsa, haberleşme masraflarıyla birlikte düşünülmelidir. Bütün stratejilerin context'in bütün durum bilgisine ihtiyacı olmayabilir. Çoğunluğun ihtiyacı var diye bilgi üst sınıftan bütün stratejilere verilirse haberleşme yükü artar. Yalnız ihtiyacı olan alt sınıflara verilirse kod karmaşıklığı biraz artar ama haberleşme masrafı düşer; hocaya göre bu daha akıllıca bir iştir.

13. Strategy ile Builder Karşılaştırması

Hoca Strategy şemasını daha önce işlenen Builder şemasıyla yan yana gösterdi. Builder örneğinde RTF okuyucu (yönetici), TextConverter ve ASCII, TeX gibi dönüştürücüler; Strategy örneğinde uygulama, SatırBölümleme ve kesmeli, kelimeden, sınırdan bölümleyiciler vardır. Ana sınıf şeması açısından iki kalıbın tasarımı aynıdır; ufak tefek sarkan kullanım ilişkileri dışında fark yoktur. Ama iç amaçları tamamen farklıdır:

- Builder'da yönetici (RTF okuyucu), ilgili dönüştürücüyü çok ince ayarlı bir denetim akışı içinde kullanır; parçaları nasıl oluşturup birleştireceğini bilir ve karmaşık bir bütünü parça parça inşa ettirir.
- Strategy'de uygulamaya bir bölmeleyici verilmiştir ve bütün sorumluluk somut stratejiye atılır: "yap bunu" denir ve problem bütüncül olarak çözdürülür. Context, işin nasıl yürüdüğünü bilmez, bilmek zorunda da değildir.

  Hocanın vurgusu: "Strategy ve Builder kalıpları arasındaki farklar nelerdir?" diye bir soru sorulabilir. Cevapta her ikisinin benzerlikleri ve farkları birlikte verilmelidir. Kalıpları farklılaştıran şey amaçları, kullanım yerleri ve gerçekleme ayrıntılarıdır (02:17:49, 02:18:10).

14. Örnek Gerçekleme: Uluslararası Vergi Hesabı

14.1 Çözüm 1: switch-case

Örnek Design Patterns Explained kitabından alınmıştır (slaytta "Kaynak: DP Explained"). Uluslararası pazara hitap eden bir kişisel finans programı yazılıyor; değişik ülkelerdeki değişik vergi kurallarıyla işlem yapılması gerekiyor. Ana program TaskController'dır.

Uygulamanın destekleyeceği ülke ya da vergi bölgeleri bir enum'da listelenir; ilk başta Türkiye için yazılmış, sonra ABD ve Avrupa Birliği eklenmek istenmiştir. TaskController'da, ödenecek vergiyi hesaplayan taxAmount metodu içinde ülke koduna bakılıp ilgili işlem yapılır; kodu ayarlamak için setCountry vardır (ekrandan, 02:20:00):

```
package strategy.solution1;
public enum Country { TR, US, EU }
public class TaskController {
    private Country code;
    public void setCountry(Country code){
        this.code = code;
    }
    public double taxAmount(double income){
        switch (code) {
        case TR: return income * 0.18;
        case EU: return income * 0.15;
        case US: return income * 0.12;
        default: return income * 0.10;
        } } }
```

Slayttaki UML'de `<<Enum>> Country (TR, US, EU)` ve `TaskController (code: Country; + taxAmount(income: double): double; + setCountry(code: Country))` vardır. Örnekte bölgeler arasındaki fark çok basit biçimde yalnız oranlardadır. Hoca, gerçekte vergi hesabının çok karmaşık olduğunu ve her case'in içinin rahatça 100, 200, 300 satır olabileceğini söyledi. İşi biraz bilen biri bunu 300 satırlık tek bir blok yerine ortalama 15 satırlık 20 metoda böler, ama her durumda devasa bir switch kodu bekliyor olur.

> **Hocanın vurgusu:** Kod yazarken kendinizi if-else if-else zinciri ya da switch-case kullanırken görüyorsanız, bu, kalıtım ve çok biçimlilik kullanan daha doğru, daha iyi bir tasarım yapabileceğiniz anlamına gelen bir uyarıdır (02:21:11).

14.2 Çözüm 2: Strategy kalıbı ile

Farklı ülkelerin farklı vergi kurallarına çalışma anında geçilebilmek istenir. Bir vergi kuralı arayüzü hazırlanır; yapılması gereken vergi işlemleri bu arayüzde tanımlanır ve her bölge için bu arayüzü gerçekleyen bir kural sınıfı kodlanır (ekrandan, 02:22:30):

```
TaskController
  tax : TaxRule
  + setTax( tax : TaxRule )
TaskController ----> <<interface>> TaxRule : + taxAmount( income : double ) : double
TaxRule'u gerçekleyenler: TRrules, EUrules, USrules
```

TaskController vergilendirme gerektiğinde yalnız bu metodu çağırır ve vergiyi hesaplatır. Farklı ülkelerin kurallarını ve ayrıntılarını bilmek zorunda kalmaz; bunlar "neyin değiştiğini bul ve sarmala" ilkesi gereğince kendi sınıflarında kodlanmıştır. Kendisine hangi TaxRule gerçeklemesi verilmişse vergi hesabını ona yaptırır. Kalıbın asıl esnekliği vergi hesabının çok daha karmaşık kurallara dayandığı gerçek durumlarda ortaya çıkar.

Vergilendirmede emlak vergisi, araç vergisi, konut vergisi gibi başka türler de varsa iki yol vardır. Bu işler kendi aralarında anlamlı bir bütün oluşturuyorsa ilgili kural sınıfına ek metotlar konur ve bu metotlar arayüzde tanımlanır. Ama gelir, araba ve ev vergisi birbirinden çok farklı işlerse, gelir vergisi için IncomeTaxRule, araç vergisi için VehicleTaxRule gibi ayrı arayüzler ve onların somut gerçeklemeleri yapılır; böylece iki farklı iş için iki ayrı strateji yapısı kurulmuş olur. Hoca, duruma göre karar verileceğini söyledi.

14.3 Somut stratejiler ve Singleton

Hoca, Dörtlü Çete'nin her bir strateji gerçeklemesinin Singleton olabileceğini söylediğini aktardı. Gerekçe: farklı ülkeler için farklı kurallar vardır ama belirli bir ülke için tek bir kural takımı vardır; Türkiye'de de Avrupa Birliği'nde de kural gelire ve miktara göre değişse de bütün olarak tektir. İstenirse her strateji gerçeklemesi Singleton yapılabilir, gerek görülmezse yapılmaz. Singleton için constructor private yapılır, sınıfa kendi türünden private static bir üye alan konur ve getInstance metodu bu alanı gerekiyorsa ilklendirip örneği döndürür.

> **Ders sonrası not:** GoF kitabında büyük olasılıkla Strategy için "strateji nesneleri çoğunlukla iyi birer Flyweight olur", State için ise "State nesneleri çoğunlukla Singleton'dır" denir. Yani Singleton önerisi kitapta State kalıbına bağlıdır. Ülke başına tek bir vergi kuralı nesnesi kullanmak yine de geçerli bir tasarım seçeneğidir.

15. State Kalıbı

15.1 Amaç ve GoF örneği

Kalıbın amacı, bir nesnenin iç durumu değiştiğinde davranışının da değişmesini sağlamaktır. Aslında bir nesnenin üye alanları değişince bu alanlarla işlem yapan metotların davranışı da bir anlamda değişir; ama hoca bu kadar küçük ölçekli değişiklikten değil, daha büyük ölçekli bir değişiklikten söz ettiğini belirtti.

Örnek: bir TCP/IP bağlantısı belli bir anda ya established, ya listening ya da closed durumundadır; aynı anda ikisi birden olamaz. Bağlantı kurulmuşsa gönderilen ve gelen paketlerin yapısı farklıdır; listening durumunda farklı, closed durumunda daha farklı işler yapılır. Bağlantı o anki durumuna göre farklı davranır. Bu yine bir switch-case ile kodlanabilir ama istenmez.

"Neyin değiştiğini bul ve sarmala" ilkesine göre değişen, TCP bağlantısının durumlarıdır. Slayttaki önerilen çözüm (ekrandan, 02:30:00):

- Neyin değiştiğini bularak sarmalamak: TCP durumları.
- Parça-bütün ilişkisini kullanmaya teşvik: TCP soketi sınıfı ve soket durumunu simgeleyen üye.

```
TCPConnection <>---state---> TCPState : Open(), Close(), Acknowledge()   (soyut)
TCPConnection : Open(), Close(), Acknowledge()
    Open() notu: state->Open()
TCPState'in alt sınıfları: TCPEstablished, TCPListen, TCPClosed
                           (her biri Open(), Close(), Acknowledge())
```

TCPConnection sınıfında soyut TCPState türünden bir state üyesi vardır. TCP bağlantısında açma, kapama ve onaylama (acknowledgement) işlemleri yapılır; bunlar soyut metot olarak verilir ve alt sınıflarda gerçeklenir. Bağlantı belli bir anda tek bir durumdadır; o anda bir open isteği gelince istek state üyesine iletilir ve o an hangi durum nesnesiyle ilişkiliyse onun gerektirdiği biçimde open, close ve acknowledge işlemleri yapılır.

15.2 Strategy'den farkı

Hoca, bu kalıbı Strategy'den ayıran şeyin durum geçişleri olduğunu anlattı. Bir durum nesnesi, gelen isteğe göre bağlantıyı başka bir duruma geçirebilir. Hocanın örneklerine göre established durumundayken close gelirse bir iş yapılmaz, belki basitçe false dönülür. Listening durumundayken close gelirse bağlantının durumu closed'a çevrilir. Closed durumundayken yine close gelirse bir şey yapılmaz.

> **Ders sonrası not:** Gerçek TCP'de Established durumundaki bir bağlantıda close istenince büyük olasılıkla bağlantıyı sonlandırma süreci başlar (FIN gönderilir ve bağlantı kapanma durumlarına geçer). Hocanın "established durumunda close bir şey yapmaz" örneği, durum nesnelerinin aynı isteğe farklı tepki verebileceğini göstermek için seçilmiş basitleştirilmiş bir senaryodur.

Strategy kalıbından farklı olarak somut durum bir state'ten yeni bir state'e geçişi sağlayabilir; Strategy'de bu yoktu. State'in sahibi (owner) olan nesnenin başka bir duruma geçmesini isteyen başkaları da olabilir; böyle isteklerin geçerli olduğu problemler için genelde State kalıbı kullanılır. Hoca, "genelde" derken fazla genelleme yapmış olabileceğini, iki türün de olabileceğini söyledi.

Kalıp bileşenleri: Context vardır; context'in içinde olabileceği çeşitli durumlar state üyesiyle desteklenir; her belirli durum bir ConcreteState alt sınıfında gerçeklenir. Durum değişikliğinden context de sorumlu olabilir, somut durumlar da. Somut durumlar geçişi yapacaksa state'ten context'e de bir ilişki vardır.

Farklar özetle şunlardır: Strategy'de strateji değişimleri genelde daha seyrektir; State'te durum değişiklikleri daha sık olabilir. Context durumu değiştikçe kendi somut durumunu seçebilir ve geçişleri açıkça kodlayabilir, ya da bu iş somut durumların içinden yapılabilir. Sınıf yapısı olarak bakıldığında iki kalıp tamamen aynıdır.

16. Örnek Gerçekleme: Kişisel Bilgi Yöneticisinde Temiz ve Kirli Durum

Hoca bu örneği çok beğenmediğini, bu dönem güncelleyemediğini, ama hiç örnek olmamasından iyi olduğunu söyledi. Bir kişisel rehber (personal information manager) programındaki kayıtların iki durumu olabilir: temiz (clean) ya da kirli (dirty). Kirli durumdayken uygulamadan çıkılmak istenirse "kaydedilmemiş değişiklikleriniz var; kaydetmek mi yoksa vazgeçmek mi istiyorsunuz?" diye sorulur ve kullanıcının cevabına göre değişiklikler kaydedilir ya da kaydedilmez. Kaydedilince durum temiz olur. Temiz durumdayken çıkılırsa ek bir soru sorulmadan çıkılır. Bunlar durum olduğu için State kalıbı kullanılır.

Soyut bir State sınıfı ve onun somut CleanState ve DirtyState alt sınıfları tanımlanır. Rehber sınıfında bir clean ve bir dirty durum nesnesi tanımlanmış, ayrıca o anki durumun hangisi olduğunu belirten normal bir üye alan vardır. Üst sınıf (ekrandan, 02:34:45):

```
package dp.state.example;
import java.io.*;
public abstract class State {
    public void save(File f, Serializable s) throws IOException {
        FileOutputStream fos = new FileOutputStream(f);
        ObjectOutputStream out = new ObjectOutputStream(fos);
        out.writeObject(s);
    }
    public void edit( ) { /*Gerekli işler*/ }
}
```

Edit işleminde olanlar farklı durumlarda farklıdır. Temiz durumdayken edit gelince uygulamanın durumu kirliye çekilir ve sonra üst sınıftaki düzenleme işlemlerinin yapılması sağlanır. Kirli durumdayken edit gelince ek bir şey yoktur; sınıfın varsayılan davranışı, yani üst sınıftaki davranış benimsenir. Kirli durumdayken save gelince kaydedilir ve durum temize çekilir.

Hoca kodu gösterirken temiz durumdayken save geldiğinde de durumun temize çekildiğini, böylece iki save metodunun tamamen aynı olduğunu fark etti. Bunu kendi hatası olarak niteledi: temiz durumdayken zaten temizdir, yeniden temize geçirmeye gerek yoktur. Kaynak kodlarda (`dp.state.example`) bunu düzelteceğini söyledi.

Örnek Sorular

Soru 1 (derste sözlü, 00:08:14): Command kalıbının menü örneğinde menülerin içinde alt menüler de olabilseydi tasarım nasıl genişletilirdi?

> **Çözüm (derste verilmedi):** Hoca yalnız "Composite ile Command birlikte kullanılmalı" dedi ve alıştırma olarak bıraktı. Menü ve menü öğesi ortak bir bileşen üst sınıfından türetilir; Menu bir composite olarak alt bileşenlerini (alt menüler ve öğeler) tutar, MenuItem ise yaprak olarak bir Command nesnesine bağlanır ve tıklanınca onun Execute() metodunu çağırır. Böylece komut bağlama Command'da, hiyerarşi Composite'te kalır.

Soru 2 (derste sözlü, 00:29:36 ve 02:17:49): Strategy ve Builder kalıplarının sınıf şemaları benzer. Bu iki kalıbın benzerlikleri ve farkları nelerdir?

Cevap: Hocanın cevabı: ana sınıf şemaları neredeyse aynıdır; bir yönetici/context soyut bir tür üzerinden somut alt sınıflardan birini kullanır. Farkları amaçları, kullanım yerleri ve gerçekleme ayrıntılarıdır. Builder'da yönetici parçaları nasıl oluşturup birleştireceğini bilir ve karmaşık bir bütünü ince ayarlı bir denetim akışıyla parça parça inşa ettirir. Strategy'de context işin nasıl yürüdüğünü bilmez; bütün sorumluluğu verilen somut stratejiye bırakır ve problemi bütüncül olarak çözdürür.

Soru 3 (derste sözlü, 02:30:25–02:32:32): Strategy ve State kalıplarının sınıf yapısı aynıdır. Aralarındaki fark nedir?

Cevap: Hocanın cevabı: State'te somut durum nesneleri context'i yeni bir duruma geçirebilir; bu yüzden state'ten context'e de bir ilişki vardır. Strategy'de stratejinin kendiliğinden başka bir stratejiye geçirmesi yoktur. Strategy'de değişimler genelde seyrek, State'te durum geçişleri daha sıktır. Geçişleri context de yapabilir, somut durumlar da.

Soru 4 (ekrandan, 01:20:00 ve 01:30:00; derste sözlü, 01:28:42): İç sınıf ile yazılan memento çözümünde, sahte bir Originator'dan (`fakeSubject`) alınan memento gerçek nesneye `setMemento` ile verilirse ne olur? Bu nasıl engellenebilir?

Cevap: Hocanın gösterdiğine göre bu çözüm bunu engelleyemiyor; sahte memento kabul ediliyor. Slayttaki öneriler: memento'ların idaresi Originator'ın kendisine verilir ve Caretaker setMemento yerine `activateMemento(int siraNo)` çağırır; ya da harici bir Caretaker isteniyorsa Originator memento oluştururken kendi hashCode'unu veya kurucuda rastgele atanan bir seri numarasını memento'ya koyar ve uyumsuz kodlu memento'yu reddeder. hashCode yalnız program çalıştığı sürece geçerlidir; rastgele seri numarası daha kalıcıdır.

Soru 5 (derste sözlü, 01:40:36): Subject ile Observer'ın farklı mekânlarda olduğu ve iletişimin ücretli olduğu bir IoT uygulamasında pull ve push modelleri arasında nasıl seçim yapılır?

Cevap: Hocanın cevabı: subject yalnız "değiştim" derse giden paket küçüktür; observer durumun yalnız bir alt kümesiyle ilgileniyorsa sadece o veriler için getter çağırır ve daha az veri taşınır (pull'un avantajı). Subject bütün durumunu gönderirse (push) gereksiz veri gidebilir ve haberleşme masrafı artar. Observer'ın sürekli "değiştin mi" diye sorması ise hem iletişimi hem işlemci gücünü boşa harcar. Çok hassas bir sensörde her küçük değişikliğin bildirilmesi de masraflıdır; haberleşme masrafı, işlemci gücü ve kod karmaşıklığı birlikte tartılarak karar verilir.

Soru 6 (ekrandan, 00:43:40; derste sözlü, 00:52:21): Iterator'lı çözümde de `printMenu` ile `printVegetarianMenu` arasında kod benzerliği kalıyor. Bu tekrar nasıl giderilebilir?

Cevap: Hocanın cevabı: `printMenu(Iterator<MenuItem>)` metoduna ikinci bir parametre (yalnız vejetaryen mi) eklenir; parametre true ise yalnız vejetaryen öğeler basılır. Böylece dolaşma ve basma kodu tek bir noktada toplanır.

Dersin Toparlaması

Bu derste davranışsal kalıplardan altısı işlendi. Command, istekleri nesneye çevirerek isteği verenle yürüteni ayırır ve kuyruklama, loglama, undo/redo gibi olanaklar sağlar. Iterator, bir bütünün iç yapısını açığa çıkarmadan dolaşılmasını sağlar; menü örneğinde kopya kodun nasıl

tek bir dolaşma metoduna indiği görüldü. Memento, durumu sarmalamayı bozmadan saklamanın yollarını (C++'ta friend, Java'da paket ya da iç sınıf, klonlama) ve bunun sınırlarını gösterdi. Observer, bir nesnedeki değişikliği bağlaşımı artırmadan ilgililere duyurur; pull ve push modellerinin avantajları birbirinin simetriğidir. Strategy, aynı işi yapan algoritmaların çalışma anında değiştirilmesini sağlar ve switch-case yerine çok biçimlilik kullanmayı önerir. State ise aynı sınıf yapısıyla, durum nesnelerinin kendi aralarında geçiş yapabilmesiyle Strategy'den ayrılır.

Dersin ortak mesajı, benzer sınıf şemalarının aynı kalıp anlamına gelmediğidir: Iterator ile Bridge, Strategy ile Builder ve Strategy ile State şema olarak benzese de amaçları ve çalışma biçimleri farklıdır. Hoca, kalan kalıplardan Template Method, Visitor ve Mediator ile devam edileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.